Advance Your R Testing Roadmap

Accelerate your feedback loops. Build tests that help you deliver faster.

Optimize Test Organization & Execution

Get feedback as fast as possible.

- [ ] Organize test files with clear, descriptive prefixes grouping related tests, e.g.: `test-plots-barchart.R`, `test-plots-boxplot.R`
- [ ] Use selective running by filtering on file prefixes for super-fast feedback during development: `testthat::test_dir("tests/testthat", filter = "plots")`

Design for Maintainability & Reliability

Convey testing intent with clear, descriptive test names and messages.

- [ ] Describe expected behavior in test titles.
- [ ] Ensure tests are independent, isolated, and reproducible. Avoid shared state or hard-to-control external effects.
- [ ] Use Arrange, Act, Assert comments for clarity and consistency.

Use Robust Test Data Management

Build a robust test suite with reliable, controlled data.

- [ ] Create reusable fixture functions with clear, consistent naming:
  `fixture_barchart <- function() { }`
  `fixture_calculate_metrics <- function() { }`
- [ ] For larger datasets, store data externally in human-readable formats (e.g., JSON, CSV) and load via fixtures.

Replace External Dependencies

Effectively isolate your code from external services.

- [ ] Use `testthat::local_mocked_bindings()` and/or the `mockery` package or dependency injection for mocks and fakes.
- [ ] Decouple business logic from external APIs and databases in your code to enable isolated, reliable tests.
- [ ] Test for success, failure, and edge case responses to cover all realistic conditions.

Build a Reliable Pipeline

Trust results from runs in CI.

- [ ] Keep your unit test suite runtime as fast as possible. (under 5 minutes)
- [ ] Use selective filtered test runs continuously in development to speed iteration cycles.
- [ ] Fix any intermittent test failures immediately to maintain trust in the pipeline.

Focus Testing on Impactful Areas

Don't chase 100% code coverage.

- [ ] Prioritize testing critical features, recent fixes, and high-risk code paths.
- [ ] Balance coverage pragmatically. Aim to test what protects your code, not every line blindly.
- [ ] Use mutation testing (with `muttest`) to evaluate test quality and reveal gaps in protection.

Acceptance Testing

Validate real user workflows from end to end.

- [ ] Describe key user journeys and expected outcomes. Write tests that validate these scenarios. Use `testthat` or adopt BDD approaches with `cucumber`
- [ ] Automate acceptance tests to detect workflow gaps early, ensuring your code delivers actual user value.
- [ ] Keep acceptance tests stable, readable, and easy to maintain alongside evolving code needs.

Embed a Testing Culture

Testing is not a solo sport. Make it part of your team's DNA.

- [ ] Add a reminder in your Pull Request (PR) template prompting developers to include tests for all new or changed features.
- [ ] Foster shared ownership of quality by encouraging your team to review and improve tests regularly.
- [ ] Monitor test suite health to quickly identify and fix flaky tests or gaps.
- [ ] Track code coverage trend for legacy codebases.

Take Action Today

A few small steps can make a big impact.

- [ ] Identify your top 3 testing bottlenecks or pain points.
- [ ] Write or improve one high-impact, fast feedback test on code you change often.
- [ ] Organize test files with domain prefixes and create helpers for selective testing.
- [ ] Automate tests locally and in CI for full, fast cycle feedback.
- [ ] Add or enhance the test reminder in your PR template to make tests part of your team's DNA.

Remember

Make tests work for you. Test what will help you move faster.

© 2025 Jakub Sobolewski